

Day 1 Evidence Workbooklet

Visible Output and Stored State

Engineering practice to computational evidence to Python output/state to support route

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

Name		Section		Date	
Score	____/39		Mastery check		secure / developing / needs support

Concrete engineering situation

Your team is doing a first-day hallway cart check. A small wheeled cart is rolled along a taped 12-meter path. One teammate times the run with a stopwatch. Another teammate enters the values in a Python notebook so the team can decide what should go into the lab record.

The problem today is not advanced physics. The problem is evidence: what did the notebook store, what did it visibly show, and which value is safe to write in the record?

Engineering task	Build a reliable evidence trail for two hallway cart trials before a result is copied into a lab record.
Computational move	Separate measurement inputs, stored variables, printed output, notebook display, and later overwrites.
Python focus	assignment, arithmetic expression, variable overwrite, <code>print</code> , final-expression notebook display, state after each cell.
Evidence to keep	exact visible lines, displayed values, stored values after each cell, a corrected lab note, and one non-mutating check.

Day 1 evidence rules

1. Assignment stores or overwrites a value. Example: `speed_mps = distance_m / time_s`.
2. `print(...)` creates visible output but does not by itself change a variable.
3. In a notebook, a bare expression at the end of a cell can display a value. Displaying a value is not the same as storing it.
4. A later assignment can overwrite an earlier stored value. Your trace must say what is stored *after each cell*, not just what looked important on the screen.

1 Read the notebook as an evidence system

[recognize](#)[predict](#)

Use the scenario above to identify what kind of evidence each line can create. Do not calculate everything yet.

```
1 # Cell 1: first run on the hallway path
2 trial_id = "cart-A"
3 distance_m = 12.0
4 time_s = 3.0
5 speed_mps = distance_m / time_s
6 print("trial", trial_id)
7 print("speed", speed_mps)
8
9 # Cell 2: second run, same path, new time
10 trial_id = "cart-B"
11 time_s = 2.4
12 speed_mps = distance_m / time_s
13 print("trial", trial_id)
14 speed_mps + 0.2
15
16 # Cell 3: keep the adjustment separate
17 adjusted_speed = speed_mps + 0.2
18 print("stored", speed_mps)
19 print("adjusted", adjusted_speed)
20
21 # Cell 4: decide to record the adjusted value
22 speed_mps = adjusted_speed
23 print("recorded", speed_mps)
```

Pts.	Prompt	My evidence
1	Which variable names store measurement inputs rather than calculated results?	
1	Which line first calculates a speed from the measured distance and time?	
1	Which line in Cell 2 displays a candidate adjusted value without storing that adjustment?	
1	Which line in Cell 4 overwrites the stored value of speed_mps?	

2 Trace stored state after each cell

[trace](#) [predict](#)

Complete the state table. If a variable does not exist yet, write `not defined`. The first row has hints; the later rows require you to track overwrites.

Checkpoint	trial_id	time_s	speed_mps	adjusted_speed	Reason for any change
After Cell 1	cart-A	3.0		not defined	
After Cell 2					
After Cell 3					
After Cell 4					

Pts.	Prompt	My response
2	At what checkpoint does <code>speed_mps</code> first become 5.0? Explain the measurement change that caused it.	
2	At what checkpoint does <code>speed_mps</code> first become 5.2? Explain what line caused the overwrite.	

3 Separate printed output from notebook display

[predict](#)[trace](#)

Visible evidence can come from printed output or from a final notebook expression. Complete both columns. If a cell has no notebook display, write **none**.

Cell	Printed output lines from <code>print(...)</code>	Notebook display from final bare expression
Cell 1		
Cell 2		
Cell 3		
Cell 4		

Common trap to check

A displayed value can look official because it appears on the screen. But a lab record should say whether the value was printed, displayed as a candidate calculation, or stored as the current value after an assignment.

Pts.	Prompt	My response
2	In Cell 2, why is the displayed 5.2 not yet the stored value of <code>speed_mps</code> ?	
2	In Cell 3, why is it useful to print both <code>stored</code> and <code>adjusted</code> ?	

4 Debug the lab record

[debug](#) [explain](#) [test](#)

A teammate writes this lab note after Cell 2:

Lab note to debug

Trial cart-B speed was recorded as 5.2 m/s because the notebook showed 5.2.

Pts.	Prompt	My response
2	What is accurate in the teammate's note?	
2	What is inaccurate or incomplete in the teammate's note?	
2	Rewrite the note so it separates measured speed, displayed candidate adjustment, and the value actually recorded after Cell 4.	

Pts.	Prompt	My response
1	Write one line you could run after Cell 2 to check the stored value of <code>speed_mps</code> without changing it.	
1	Write one line you could run after Cell 3 to check whether <code>adjusted_speed</code> exists.	
2	Before calculating speed, what measurement condition should the team check so the result is physically meaningful? Write it in plain language.	

5 Compare two possible edits

[debug](#)[implement](#)[explain](#)

The team considers two ways to apply the 0.2 m/s adjustment. Both can show 5.2; they do not leave the same evidence trail.

Version A: keep candidate separate

```
adjusted_speed = speed_mps + 0.2
print("stored", speed_mps)
print("adjusted", adjusted_speed)
```

Version B: overwrite immediately

```
speed_mps = speed_mps + 0.2
print("stored", speed_mps)
print("adjusted", speed_mps)
```

Pts.	Prompt	My response
1	In Version A, what is stored in <code>speed_mps</code> after the code runs?	
1	In Version A, what is stored in <code>adjusted_speed</code> after the code runs?	
1	In Version B, what is stored in <code>speed_mps</code> after the code runs?	
2	Which version gives a better audit trail for a lab record? Explain your choice.	
2	If the goal is to record the adjusted value only after review, write one sentence explaining why Cell 4 is a safer place to overwrite <code>speed_mps</code> .	

6 Mastery check and support route

[transfer](#)[explain](#)

Use your own evidence from the workbooklet. Do not write a generic answer.

Pts.	Prompt	My response
2	What is one example from this notebook where the screen showed a value that was not yet the stored value that would be recorded?	
2	In one or two sentences, explain why engineers should preserve both visible output and stored-state evidence during early computational checks.	

My mastery check

Mark the strongest statement that is true after this workbooklet.

☐	Secure: I can trace stored state and visible output separately, including a displayed value that is not stored yet.
☐	Developing: I can calculate some values, but I still need support separating display, print, assignment, and overwrite.
☐	Needs support: I need a smaller example before I can explain what Python stored after each cell.

My next support move

My issue is mostly: setup / Python syntax / tracing / arithmetic / notebook display / confidence / time.
The first place I got uncertain was: _____
My next small action is: ask a partner / ask instructor / rerun a cell / rebuild the trace table / try the recovery version.

Workbooklet target: I can build an evidence trail that separates measurement inputs, stored variable state, printed output, notebook display, and a later overwrite before a result enters an engineering record.

Copyright © 2026 Lance L.A. White. All rights reserved.